

Phase-Adaptive Generation: Difficulty-Aware Semi-Autoregressive Decoding for Diffusion Language Models

Keshav Krishna Prajesh Kumar Siva Satvik Mandapati

New York University
Building LLM Reasoners — Spring 2026
`{kk6081, pg2973, sm12779}@nyu.edu`

Abstract

Phase-Adaptive Generation (PAG) is an original research project on difficulty-aware decoding for diffusion language models. The central hypothesis is that generated text follows recoverable “difficulty phases” that can be segmented from token-level confidence trajectories and used to predict the next control tuple for decoding. Our current pipeline builds fixed-decoding traces, segments them with change-point detection over stabilizing token signals, and trains a Transformer next-segment predictor over tuples that encode block size and refinement steps, with support for extensible fields. This paper-style writeup describes the problem setting, the data pipeline, the segmentation and prediction method, and the current implementation status of the project.

1 Introduction

Autoregressive (AR) language models generate one token at a time and therefore pay a latency cost that becomes significant on long reasoning traces. Diffusion-based LLMs (dLLMs) such as Dream 7B [Ye et al., 2025] and LLaDA [Nie et al., 2025] reduce this cost by generating in blocks through iterative denoising, but existing decoding policies are typically uniform: the same chunk size and refinement budget are applied regardless of whether the model is in an easy completion region or a difficult reasoning region.

Our project studies whether these generation traces contain reusable structure that can support better scheduling. The working hypothesis is that dLLM outputs move through difficulty phases: some spans stabilize quickly, while others remain uncertain and require more refinement. If those phases can be detected early enough, a scheduler can allocate compute more intelligently. In our current formulation, PAG aims to use phase predictions to adapt both chunk size and refinement intensity.

The writeup and codebase are organized to support that question in stages. We first collect traces from a baseline decoder, then segment the traces with change-point detection, then turn each segment into a tuple sequence, and finally predict the next segment tuple from previous ones. This staged setup is intentionally testable so that each step can yield intermediate evidence even before a full adaptive decoder is completed.

2 Related Work

Diffusion LLM decoding. Dream 7B [Ye et al., 2025] and LLaDA [Nie et al., 2025] show that diffusion-style decoding can be competitive with AR baselines on reasoning and planning tasks. Block Diffusion [Arriola et al., 2025] frames semi-autoregressive generation as a bridge between AR and diffusion decoding.

Adaptive block inference. AdaBlock-dLLM [Lu et al., 2025] is the closest prior idea to our project because it adapts block size based on confidence-like signals. Our project differs in two ways. First, we treat segmentation as an explicit learning problem over trace signals rather than only as a reactive confidence heuristic. Second, we predict a richer control tuple that can later drive both block size and refinement intensity.

Why this formulation matters. In the repository, the phase-analysis stage already exposes token-level traces, stabilizing features, and segment summaries. That makes the project a natural fit for a learned next-segment model: the model can operate on symbolic segment tuples rather than raw text, which keeps the method lightweight and interpretable.

3 Method

3.1 Pipeline Overview

The pipeline used in the repository is: baseline decoding $\rightarrow$ trace collection $\rightarrow$ phase segmentation $\rightarrow$ tuple construction $\rightarrow$ next-segment prediction $\rightarrow$ tuple-conditioned unmasking.

The baseline stage produces token-level traces and per-step token signals. The phase stage converts those traces into phase annotations and predictor-ready examples. The predict stage then uses those segment sequences to learn a next-tuple for generation.

3.2 Step 1: Baseline Trace Collection

The first scheme point is to generate baseline traces with per-step token probabilities and stabilization information. In the current codebase, the baseline stage records each token’s refinement history, including top-1 probability, top-2 probability, selected logit, and extra scalar values such as entropy. The important conceptual output is a token trajectory over refinement steps, which allows us to ask when each token becomes stable.

This trace representation is essential because phase prediction depends on the dynamics of confidence, not just the final decoded text. The repository already stores these traces in a unified JSON schema so that downstream analysis can consume them without depending on a specific inference backend.

3.3 Step 2: Change-Point Detection and Segment Extraction

The second scheme point is to sweep detection sensitivity and run change-point detection to obtain segment candidates. In practice, the code uses stabilizing scalar features such as entropy, margin, or top-1 probability, then applies PELT over the resulting series. The detector produces breakpoints that partition the token sequence into contiguous segments.

For each segment, we compute summary statistics and derive the segment-level control variables. In the current extraction pipeline, each segment tuple stores (block_size, refinement_steps), where block size is the number of tokens in the segment and refinement steps are obtained from token-level observation histories (per-token counts aggregated at the segment level). Optional additional fields can be appended later, such as stabilization statistics, segment-level uncertainty measures, or scheduler-specific controls.

3.4 Step 3: Tuple Sequences and Next-Segment Prediction

The third scheme point is to represent each segment as an ordered tuple and predict the next tuple from the current history. In the current implementation, a tuple is represented as (block_size, refinement_steps, ...). The representation is intentionally extensible through the shared schema so additional integer control fields can be added with minimal code changes.

We currently use a compact Transformer encoder predictor in `phase_predict`. Given a fixed window of previous tuples, the model applies self-attention over the full context and regresses the next tuple fields, which are then rounded to non-negative integers at inference time. This design retains parallel training efficiency, captures longer-range tuple interactions than a fixed-order heuristic model, and supports future extension to richer tuple schemas.

3.5 Step 4: Tuple-Conditioned Diffusion Unmasking

The fourth step is to consume the predicted control tuple as an inference-time command for the diffusion decoder. Concretely, once the model predicts a tuple like $(b, \tau, \dots)$, we interpret it as scheduling parameters for the next decode action: block size b determines how many masked positions to unmask in the next chunk, and refinement budget τ determines how many denoising/refinement iterations are spent on that chunk.

Operationally, the decoder selects the next block according to the predicted block size, applies diffusion unmasking over those positions, and runs exactly the tuple-specified refinement schedule before committing tokens. The

resulting partially decoded sequence is then fed back into the same loop, producing an updated tuple history for the next prediction. This closes the control loop from phase prediction to token generation and makes tuple prediction directly actionable for adaptive decoding.

4 Implementation Status

The repository already contains the building blocks for the final system. The `phase_cpd` package handles trace collection, feature extraction, and segmentation. The new `phase_predict` package adds tuple extraction utilities, a Transformer model, training and prediction wrappers, checkpointing, and an end-to-end training script (`scripts/train_phase_predict.py`) for command-line experimentation on real traces. Tests cover schema validation, dataset/window construction, extraction utilities, model forward passes, training behavior, and prediction/checkpoint round-trips, which gives us a correctness baseline before larger-scale experiments.

This stage is therefore more than a sketch: the data pipeline exists, the tuple encoding is implemented, and the next-segment predictor can be trained on extracted segment sequences. What remains for the final project is to connect the predictor more tightly to full adaptive scheduling and report quantitative results on held-out traces.

5 Preliminary Analysis and Limitations

At the moment, the strongest evidence we have is architectural rather than benchmark-based: the pipeline is modular, the tuple representation is interpretable, and the next-segment task is now modeled by an explicitly sequence-aware neural architecture. The main limitation is that we have not yet run a full held-out evaluation of next-segment accuracy and downstream scheduling gains across broad benchmark settings.

Another limitation is that the current tuple uses only block size and refinement steps, which is a compact summary rather than a full representation of decoding state. That is acceptable for a first report because it keeps the problem well defined, but the representation can be expanded if later ablations show that additional context is needed.

6 Conclusion

PAG frames phase-aware decoding as a prediction problem over segment tuples derived from dLLM traces. The current codebase supports the first four scheme points at the design level: trace collection, change-point segmentation, next-tuple prediction, and tuple-conditioned unmasking control. That gives us a clear experimental path toward evaluating whether phase-aware scheduling can improve the quality–latency trade-off for diffusion language models.

References

- J. Ye, Z. Xie, L. Zheng, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong. Dream 7B: Diffusion large language models. *arXiv:2508.15487*, 2025.
- S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li. Large language diffusion models. *arXiv:2502.09992*, 2025.
- G. Lu, H. M. Chen, Y. Karashima, Z. Wang, D. Fujiki, and H. Fan. AdaBlock-dLLM: Semantic-aware diffusion LLM inference via adaptive block size. *ICLR*, 2026.
- M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han, S. S. Sahoo, and V. Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *ICLR (Oral)*, 2025.
- C. Wu, H. Zhang, S. Xue, Z. Liu, S. Diao, L. Zhu, P. Luo, S. Han, and E. Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *ICLR*, 2026.
- S. Gong, S. Agarwal, Y. Zhang, J. Ye, L. Zheng, M. Li, C. An, P. Zhao, W. Bi, J. Han, H. Peng, and L. Kong. Scaling diffusion language models via adaptation from autoregressive models. *ICLR*, 2025.